

 TALLER 1 - UNIDAD 2: Patrones Estructurales	Departamento: Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC: 30723 Fecha: 21/05/2026
	Integrantes Grupo 3: • Ayuquina Danny. • Cañarte Saray. • Villagómez Doménica.

PATRONES ESTRUCTURALES

1. Adapter (Adaptador)

1.1. Conceptos y Definiciones

El patrón adaptador es un componente de software diseñado para resolver el problema de la incompatibilidad de interfaces entre dos componentes que se desean reutilizar o integrar en un sistema. Su función principal es actuar como intermediario o "código pegamento" (glue code) que reconcilia las diferencias entre la interfaz "requiere" de un componente y la interfaz "proporciona" de otro.

Puede funcionar como un proxy, permitiendo que un componente acceda a los servicios de otro aunque sus firmas de método o parámetros no coincidan originalmente.

1.2. Características

- Traducción de interfaces: Convierte los nombres de las operaciones, los tipos de datos y el número de parámetros de una interfaz a otra.
- Independencia: Permite que los componentes sigan siendo independientes, ya que no es necesario modificar el código fuente de los componentes originales para que funcionen juntos.
- Composición adaptable:
 - Composición secuencial: Toma el resultado de un componente, lo procesa y lo entrega como entrada al siguiente.
 - Composición jerárquica: Un componente llama directamente al adaptador como si fuera el componente final, y este redirige la llamada adecuadamente.
- Configurabilidad: En sistemas COTS (*Commercial Off-The-Shelf* o productos comerciales listos para usar), los adaptadores son esenciales para convertir representaciones de datos entre aplicaciones de diferentes proveedores que usan estructuras únicas.

1.3. Ejemplo implementado

En el sistema CRUD de estudiantes, el problema surge porque el sistema nuevo espera trabajar con una interfaz moderna llamada RepositorioEstudiante, pero existe un sistema heredado (SistemaLegacyEstudiantes) que utiliza métodos distintos y nombres incompatibles.

El adaptador (AdaptadorRepositorio) traduce las llamadas del sistema nuevo hacia el sistema antiguo.

Elemento Adapter	Clase del Proyecto	Responsabilidad
Target	RepositorioEstudiantes	Interfaz esperada por el sistema moderno
Adaptee	SistemaLegacyEstudiantes	Sistema heredado incompatible
Adapter	AdaptadorRepositorio	Traduce llamadas entre interfaces
Client	ControlEstudiante	Usa la interfaz Target sin conocer el sistema legacy

El sistema moderno espera métodos como:

- *guardar(estudiante)*
- *buscarPorId(id)*

Pero el sistema antiguo posee:

- *insertarRegistro()*
- *obtenerRegistro()*

El adaptador convierte una interfaz en otra para que el sistema moderno funcione sin modificar el sistema legacy.

1.4. Modelo de Clases

```
@startuml
skinparam classAttributeFontSize 0
skinparam shadowing false
skinparam linetype ortho

title Sistema CRUD de Estudiantes - Patrón Adapter

interface RepositorioEstudiante {
    + guardar(e: Estudiante): void
    + buscarPorId(id: String): Estudiante
}

class SistemaLegacyEstudiantes <<legacy>> {
    + insertarRegistro(codigo: String, nombre: String, edad: int): void
    + obtenerRegistro(codigo: String): Estudiante
    + borrarRegistro(codigo: String): void
}

class AdaptadorRepositorio <<adapter>> {
```

```
    - sistemaLegacy: SistemaLegacyEstudiantes
    + guardar(e: Estudiante): void
    + buscarPorId(id: String): Estudiante
    + eliminar(id: String): void
}

class ControlEstudiante <<control>> {
    - repositorio: RepositorioEstudiante
    + agregar(e: Estudiante): void
    + buscar(id: String): Estudiante
    + eliminar(id: String): void
}

class Estudiante <<entity>> {
    - id: String
    - nombre: String
    - edad: int

    + getId(): String
    + getNombre(): String
    + getEdad(): int
}
```

RepositorioEstudiante <|..
AdaptadorRepositorio

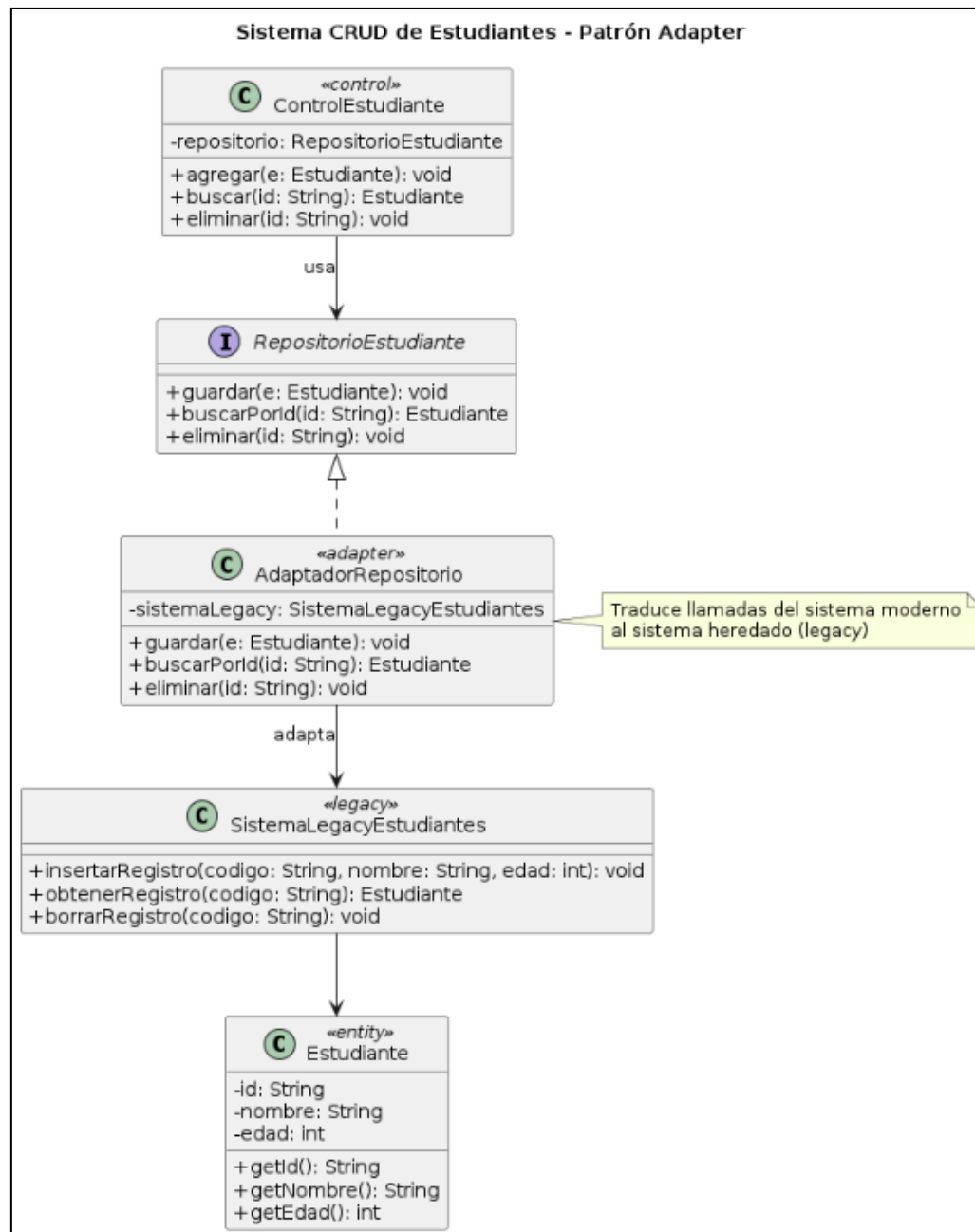
AdaptadorRepositorio -->
SistemaLegacyEstudiantes : adapta

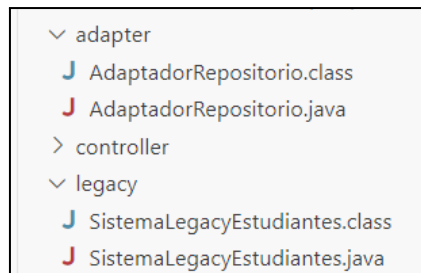
ControlEstudiante -->
RepositorioEstudiante : usa

SistemaLegacyEstudiantes -->
Estudiante

note right of AdaptadorRepositorio
Traduce llamadas del sistema
moderno
al sistema heredado (legacy)
end note

@enduml





```

===== CRUD ESTUDIANTE =====
1. Guardar
2. Buscar
3. Eliminar
4. Salir
Seleccione: 1
ID: 001
Nombre: Domenica Villagomez
Edad: 21
[ADAPTER] Convirtiendo objeto ? texto legacy
[LEGACY] Registro guardado en texto plano

===== CRUD ESTUDIANTE =====
1. Guardar
2. Buscar
3. Eliminar
4. Salir
Seleccione: 2
Ingrese ID: 001
[ADAPTER] Convirtiendo texto legacy ? objeto
[LEGACY] Buscando texto plano...
Encontrado: ID: 001 | Nombre: Domenica Villagomez | Edad: 21

```

1.5. ***Ventajas y desventajas***

- **Ventajas Generales**
 - Reutilización de software: Permite integrar componentes o sistemas existentes sin reescribirlos.
 - Compatibilidad de interfaces: Resuelve incompatibilidad de parámetros, nombres de operación y operaciones incompletas.
 - Mantenibilidad e independencia: Permite que la implementación de un componente cambie sin afectar al resto del sistema.
 - Integración de sistemas heredados: Facilita conectar tecnologías antiguas con software moderno.
- **Desventajas Generales**
 - Sobrecarga de rendimiento (Overhead): Añade niveles adicionales de interpretación y procesamiento de mensajes, lo que puede afectar la velocidad del sistema.
 - Complejidad del sistema: Hace que el sistema sea más complejo de entender y verificar.
 - Dependencia externa: Si cambia la interfaz de un componente externo, el adaptador puede dejar de funcionar.
 - Dificultades en la validación: No asegura que el componente reutilizado funcione bien en el nuevo entorno.
 - Conflicto entre adaptabilidad y diseño: Más flexibilidad puede significar menor rapidez o fiabilidad.

1.6. Conclusión y Reflexión

La implementación del patrón Adapter en el CRUD de estudiantes permitió integrar un sistema legacy con una estructura moderna sin modificar la lógica principal del proyecto. Mediante el adaptador, fue posible traducir las operaciones del repositorio actual hacia métodos incompatibles del sistema antiguo, manteniendo desacopladas ambas arquitecturas. Esta solución demuestra cómo los patrones de diseño facilitan la reutilización de código existente y mejoran la flexibilidad del software.

El desarrollo de esta implementación evidenció la importancia de separar responsabilidades y mantener una arquitectura organizada. El patrón Adapter no solo resuelve problemas de compatibilidad, sino que también reduce el impacto de futuros cambios en el sistema. Además, permitió comprender cómo integrar tecnologías o componentes heredados en aplicaciones modernas sin afectar el funcionamiento del CRUD original, favoreciendo la mantenibilidad y escalabilidad del proyecto.

2. Decorator (Decorador)

2.1. Conceptos y Definiciones

El patrón estructural Decorator (decorador), catalogado por Sommerville como uno de los patrones de la "Banda de los Cuatro", se define fundamentalmente por su propósito de permitir la extensión de la funcionalidad de una clase existente durante el tiempo de ejecución.

Este patrón funciona como una solución probada a problemas recurrentes que, en lugar de ofrecer un componente rígido, fomenta la reutilización de conceptos de alto nivel, lo cual permite que la idea del diseño se adapte con flexibilidad a las necesidades y algoritmos específicos del sistema que se está construyendo.

2.2. Características

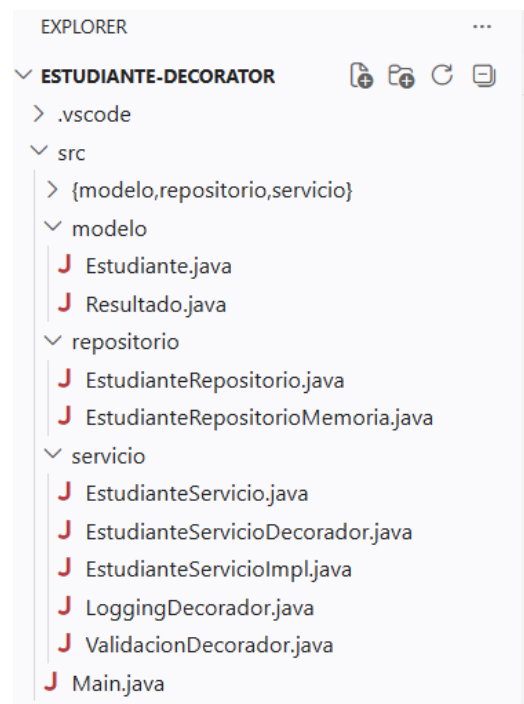
- Extensibilidad Dinámica: A diferencia de la herencia estática, este patrón se enfoca en agregar capacidades a los objetos mientras el sistema se está ejecutando.
- Abstracción de Alto Nivel: Como patrón de diseño, soporta la reutilización de conceptos. Esto significa que, aunque la idea es estándar, la implementación se adapta a las necesidades específicas del sistema en desarrollo, evitando las restricciones de los componentes ejecutables rígidos.
- Independencia: Los patrones permiten que las ideas de diseño se separen de los algoritmos particulares o tipos de datos específicos usados en las interfaces.

2.3. Ejemplo implementado

El patrón Decorator permite añadir responsabilidades adicionales a un objeto de manera dinámica, sin modificar su clase base. En el contexto del CRUD de estudiantes, permite envolver los servicios con capas de validación, logging y auditoría de forma transparente e intercambiable.

Elemento del patrón Decorator	Clase en el proyecto	Responsabilidad

Component (interfaz)	EstudianteServicio	Define las operaciones CRUD que todos los servicios deben implementar: agregar, actualizar, eliminar y listar.
Concrete Component	EstudianteServicioImpl	Implementación base del CRUD real; delega la persistencia a EstudianteRepositorio.
Base Decorator	EstudianteServicioDecorador	Clase abstracta que envuelve un EstudianteServicio y delega todas las operaciones al componente interno.
Concrete Decorator 1	ValidacionDecorador	Añade validación de datos (ID, nombre y edad) antes de ejecutar cualquier operación del servicio base.
Concrete Decorator 2	LoggingDecorador	Añade registro de auditoría (log) antes y después de cada operación CRUD.
Client	Main / EstudianteUI	Construye la cadena de decoradores y utiliza la interfaz EstudianteServicio sin conocer los detalles internos.



```

=====
SISTEMA CRUD DE ESTUDIANTES - Patron Decorator
=====

1. Agregar estudiante
2. Actualizar estudiante
3. Eliminar estudiante
4. Listar todos
5. Salir

Elige una opcion: 1

=====
AGREGAR ESTUDIANTE
=====
ID      : E001
Nombre  : Saray Canarte
Edad    : 21
[LOG 2026-05-26 09:21:49] Operación: AGREGAR      | id=E001
[LOG] Resultado AGREGAR    -> [OK]   Estudiante agregado: Saray Canarte
-> [OK]   Estudiante agregado: Saray Canarte

```

2.4. Modelo de Clases

```

@startuml
skinparam classAttributeFontSize 0
skinparam shadowing false

interface EstudianteServicio {
    + agregar(e: Estudiante): Resultado
    + actualizar(e: Estudiante): Resultado
    + eliminar(id: String): Resultado
    + listarTodos(): List<Estudiante>
}

class EstudianteServicioImpl <<control>> {
    - repositorio: EstudianteRepositorio
    + agregar(e: Estudiante): Resultado
    + actualizar(e: Estudiante): Resultado
    + eliminar(id: String): Resultado
    + listarTodos(): List<Estudiante>
}

abstract class EstudianteServicioDecorador
<<decorator>> {
    # servicio: EstudianteServicio
    + EstudianteServicioDecorador(s: EstudianteServicio)
    + agregar(e: Estudiante): Resultado
    + actualizar(e: Estudiante): Resultado
    + eliminar(id: String): Resultado
    + listarTodos(): List<Estudiante>
}

class ValidacionDecorador <<decorator>> {
    + agregar(e: Estudiante): Resultado
    + actualizar(e: Estudiante): Resultado
    + eliminar(id: String): Resultado
    - validarDatos(e: Estudiante): boolean
}

class LoggingDecorador <<decorator>> {
    + agregar(e: Estudiante): Resultado
    + actualizar(e: Estudiante): Resultado
    + eliminar(id: String): Resultado
    + listarTodos(): List<Estudiante>
}

interface EstudianteRepositorio {
    + existId(id: String): boolean
    + guardar(e: Estudiante): void
    + buscarPorId(id: String): Estudiante
    + actualizar(e: Estudiante): void
    + eliminar(id: String): void
    + listarTodos(): List<Estudiante>
}

class EstudianteRepositorioMemoria
<<repository>> {
    - lista: List<Estudiante>
    + existId(id: String): boolean
    + guardar(e: Estudiante): void
    + buscarPorId(id: String): Estudiante
    + actualizar(e: Estudiante): void
    + eliminar(id: String): void
    + listarTodos(): List<Estudiante>
}

class Estudiante <<entity>> {
    - id: String
    - nombre: String
    - edad: int
    + getId(): String
    + getNombre(): String
    + getEdad(): int
}

class FormularioCrudEstudiante
<<boundary>> {
    + clickAgregar()
    + clickActualizar()
    + clickEliminar()
    + clickMostrarTodo()
}

```

```

+ mostrarMensaje(msg: String)
}

```

```

EstudianteServicio <|..
EstudianteServicioImpl
EstudianteServicio <|..
EstudianteServicioDecorator
EstudianteServicioDecorator <|--
ValidacionDecorator
EstudianteServicioDecorator <|--
LoggingDecorator
EstudianteServicioDecorator o-->
EstudianteServicio : wraps
EstudianteServicioImpl -->
EstudianteRepositorio : usa
EstudianteRepositorio <|..
EstudianteRepositorioMemoria

```

```

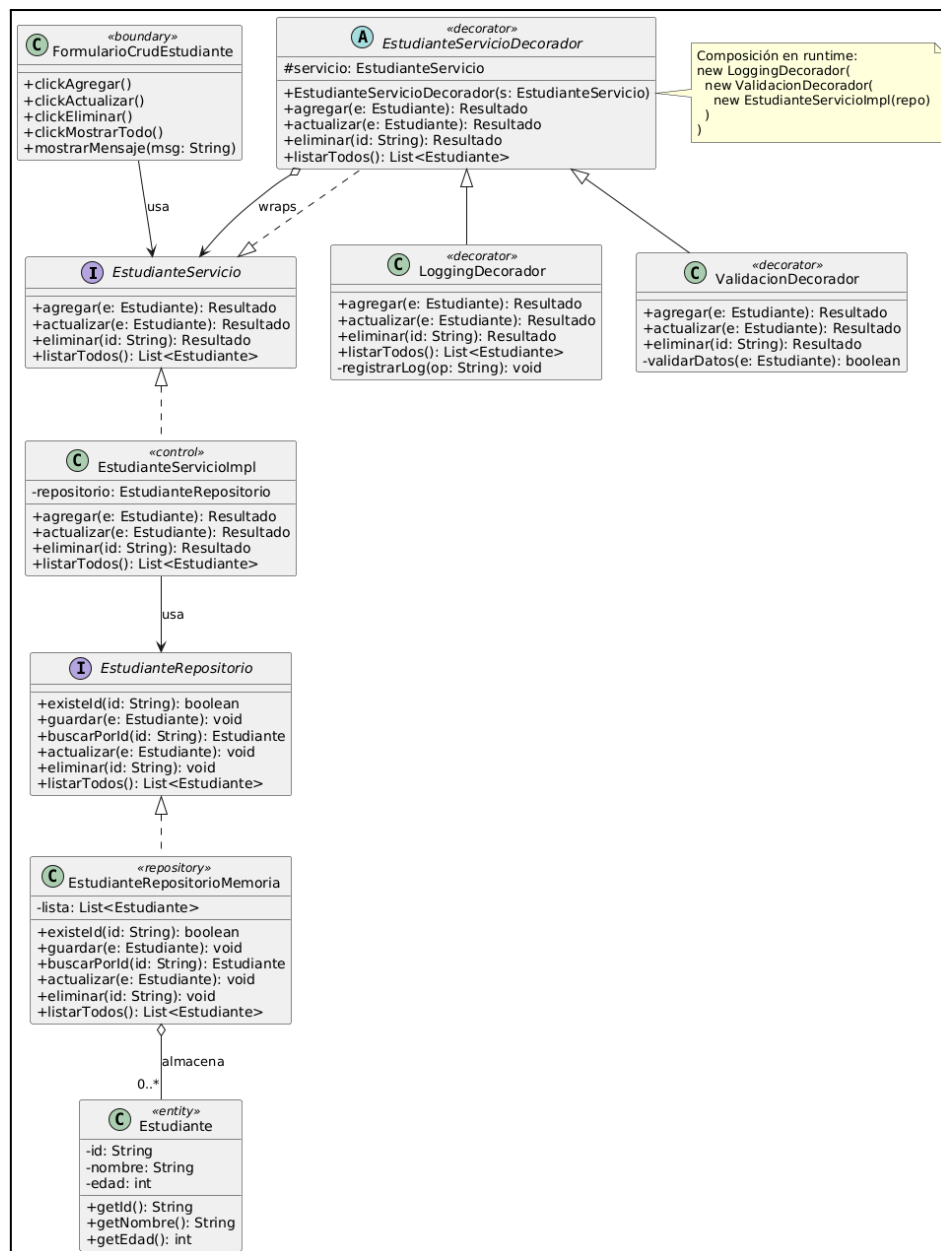
EstudianteRepositorioMemoria o-- "0..*"
Estudiante : almacena
FormularioCrudEstudiante -->
EstudianteServicio : usa

```

```

note right of EstudianteServicioDecorator
Composición en runtime:
new LoggingDecorator(
  new ValidacionDecorator(
    new EstudianteServicioImpl(repo)
  )
)
end note
@enduml

```



2.5. Ventajas y desventajas

- Ventajas:

- Reutilización de Sabiduría: Aprovecha soluciones que han sido ensayadas y puestas a prueba en diferentes entornos.
- Flexibilidad: Permite adaptar la implementación de la idea a las circunstancias locales del software.
- Mantenibilidad: Al utilizar patrones conocidos, el diseño es más fácil de explicar y comprender para otros desarrolladores.
- Desventajas:
 - Necesidad de Experiencia: Los programadores inexpertos pueden tener dificultades para decidir si es mejor reutilizar un patrón o desarrollar una solución a medida.
 - Complejidad en la Selección: Ante la existencia de cientos de patrones, puede ser complicado identificar cuál es el adecuado si el problema no encaja exactamente en los patrones de propósito general.

2.6. Conclusión y Reflexión

El sistema CRUD de estudiantes necesita, más allá de las operaciones básicas, capas transversales como validación de datos y auditoría de acciones. En lugar de mezclar esas responsabilidades dentro de `EstudianteServicioImpl` (violando el principio de responsabilidad única), el Decorator permite apilarlas de forma modular en tiempo de ejecución: `LoggingDecorator` envuelve a `ValidacionDecorator`, que a su vez envuelve a `EstudianteServicioImpl`. Cada capa hace exactamente una cosa y el `FormularioCrudEstudiante` trabaja siempre contra la misma interfaz `EstudianteServicio`, sin saber cuántas capas hay debajo.

El mayor beneficio del Decorator en este proyecto es que permite extender el comportamiento del servicio sin tocar su implementación base. Si en un futuro el sistema requiere auditoría, caché o control de acceso, basta con escribir un nuevo decorador y apilarlo, sin modificar una sola línea de `EstudianteServicioImpl`. Además, la validación y el logging quedan en clases propias con responsabilidad única, lo que hace el código más limpio y fácil de probar de forma aislada.

Sin embargo, el costo de esta flexibilidad es que la construcción del objeto se vuelve más verbal y el orden de apilado importa, lo que puede confundir a quien mantiene el código si no hay documentación clara. La depuración también se complica: una llamada a `agregar()` atraviesa silenciosamente dos o tres capas antes de llegar al repositorio, y sin un buen sistema de logging es difícil saber en qué capa falló algo. Finalmente, si no hay disciplina del equipo, existe el riesgo de que tanto el decorador como la implementación base terminen repitiendo lógica de validación, anulando precisamente la ventaja que el patrón ofrece.

3. Facade (Fachada)

3.1. Conceptos y Definiciones

El patrón estructural Facade (fachada), se define conceptualmente como una solución para ordenar las interfaces de un conjunto de objetos relacionados que a menudo han crecido en complejidad debido al desarrollo incremental. Este patrón funciona como un mecanismo de reutilización de conceptos de alto nivel, permitiendo que los diseñadores apliquen una solución probada para gestionar la interacción con subsistemas complejos a través de una interfaz unificada. Al implementar una "fachada", se facilita la comprensión y mantenibilidad del sistema, ya que se separa la complejidad interna del subsistema de la vista simplificada que se ofrece a los clientes o a otros componentes.

3.2. Características

- Organización y Ordenamiento: Su función distintiva es ordenar las interfaces de un conjunto de objetos relacionados que han ganado complejidad, usualmente debido a un proceso de desarrollo incremental.
- Simplificación de Subsistemas: Actúa como una interfaz unificada que permite interactuar con subsistemas complejos a través de una vista más simple y comprensible.
- Reutilización de Conceptos: Como patrón de diseño, soporta la reutilización de ideas y estrategias de alto nivel, lo que permite adaptar la implementación física a las necesidades específicas del sistema en lugar de usar componentes rígido
- Abstracción de la Complejidad: Oculta los detalles de implementación de los objetos individuales que la componen, facilitando la mantenibilidad y evolución del sistema al reducir el acoplamiento directo entre el cliente y las clases internas

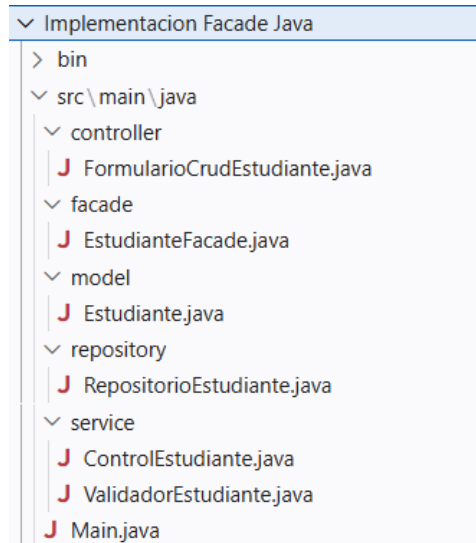
3.3. Ejemplo implementado

El patrón Facade introduce una clase que unifica y simplifica el acceso a los subsistemas internos. Aplicado a la estructura del proyecto del CRUD de estudiantes quedaría de la siguiente forma:

Elemento Facade	Clase del Proyecto	Responsabilidad
Facade	EstudianteFacade	Punto de entrada único. Expone métodos simples (agregar, actualizar, eliminar, mostrarTodos) que internamente coordinan el control, validación y persistencia.
Subsistema 1 - Validación	ValidatorEstudiante	Verifica que ID, Nombre y Edad sean datos válidos y no vacíos antes de cualquier operación. Extraído de ControlEstudiante.
Subsistema 2 - Lógica de negocio	ControlEstudiante	Orquesta las operaciones de negocio: delega la validación al validador y la persistencia al repositorio.
Subsistema 3 - Persistencia	RepositorioEstudiante	Gestiona el almacenamiento en memoria/BD: guardar, buscar, actualizar, eliminar y listar estudiantes.
Entidad del dominio	Estudiante	Representa el modelo de datos con atributos id, nombre y edad. Sin lógica de negocio.
Cliente (Boundary)	FormularioCrudEstudiante	La UI. Solo habla con EstudianteFacade, sin conocer ningún subsistema. Esto es lo que el patrón garantiza.

La diferencia clave respecto al diseño original, es que FormularioCrudEstudiante antes llamaba directamente a ControlEstudiante. Con Facade, la IU solo conoce a EstudianteFacade, que internamente coordina todo.

Capturas de pantallas:



```
=====
                        CRUD DE ESTUDIANTES - PATRÓN FACADE
=====
1. Agregar nuevo estudiante
2. Actualizar estudiante
3. Eliminar estudiante
4. Mostrar todos los estudiantes
5. Buscar estudiante por ID
6. Salir
=====
Seleccione una opción: 1

--- Agregar Nuevo Estudiante ---
Ingrese ID: L00213456
Ingrese Nombre: Danny
Ingrese Edad: 21
? Estudiante agregado exitosamente

=====
                        CRUD DE ESTUDIANTES - PATRÓN FACADE
=====
1. Agregar nuevo estudiante
2. Actualizar estudiante
3. Eliminar estudiante
4. Mostrar todos los estudiantes
5. Buscar estudiante por ID
6. Salir
```

3.4. Modelo de Clases

```
@startuml
skinparam classAttributeFontSize 0
skinparam shadowing false
skinparam linetype ortho
```

```
title Sistema CRUD de Estudiantes -
Patrón Facade
```

```
class "FormularioCrudEstudiante"
<<boundary>> {
- txtId: String
```

```

- txtNombre: String
- txtEdad: String
+ clickAgregar()
+ clickActualizar()
+ clickEliminar()
+ clickMostrarTodo()
+ mostrarMensaje(mensaje: String)
+ mostrarTabla(estudiantes:
List<Estudiante>)
}

class "EstudianteFacade" <<facade>> {
- control: ControlEstudiante
- validador: ValidadorEstudiante
- repositorio: RepositorioEstudiante
+ agregar(id: String, nombre: String,
edad: int): String
+ actualizar(id: String, nombre: String,
edad: int): String
+ eliminar(id: String): String
+ mostrarTodos(): List<Estudiante>
}

class "ValidadorEstudiante"
<<subsystem>> {
+ validarDatos(id: String, nombre: String,
edad: int): boolean
+ validarId(id: String): boolean
+ validarEdad(edad: int): boolean
}

class "ControlEstudiante" <<subsystem>>
{
- validador: ValidadorEstudiante
- repositorio: RepositorioEstudiante
+ procesarAgregar(id: String, nombre:
String, edad: int): String
+ procesarActualizar(id: String, nombre:
String, edad: int): String
+ procesarEliminar(id: String): String
+ obtenerTodos(): List<Estudiante>
}

class "RepositorioEstudiante"
<<subsystem>> {
+ existeId(id: String): boolean
+ guardar(estudiante: Estudiante): void

```

```

+ buscarPorId(id: String): Estudiante
+ actualizar(estudiante: Estudiante): void
+ eliminar(id: String): void
+ listarTodos(): List<Estudiante>
}

```

```

class "Estudiante" <<entity>> {
- id: String
- nombre: String
- edad: int
+ Estudiante(id: String, nombre: String,
edad: int)
+ getId(): String
+ getNombre(): String
+ getEdad(): int
}

```

' Relaciones
FormularioCrudEstudiante -->
EstudianteFacade : usa (solo conoce la fachada)

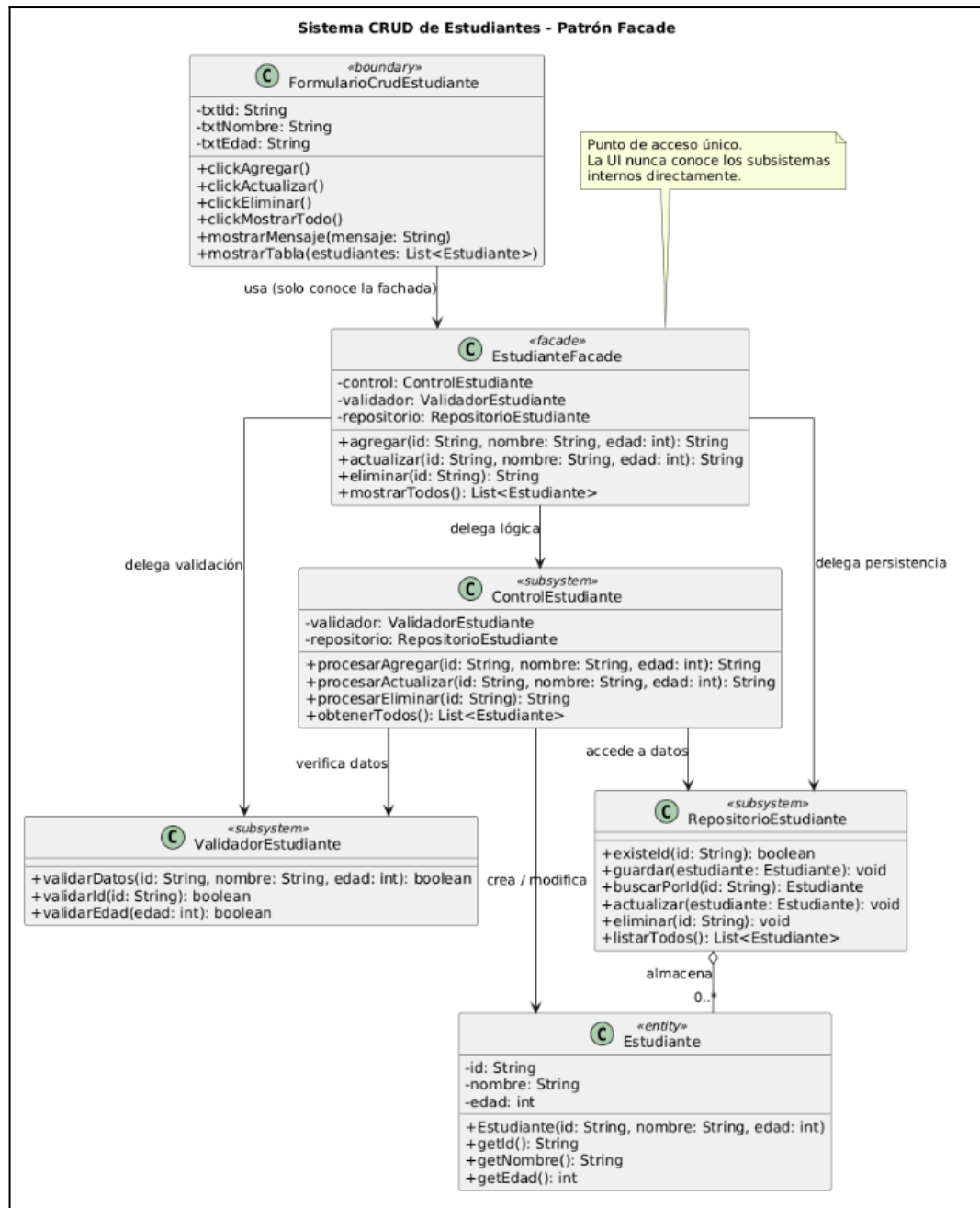
EstudianteFacade --> ValidadorEstudiante
: delega validación
EstudianteFacade --> ControlEstudiante :
delega lógica
EstudianteFacade -->
RepositorioEstudiante : delega
persistencia

ControlEstudiante --> ValidadorEstudiante
: verifica datos
ControlEstudiante -->
RepositorioEstudiante : accede a datos
ControlEstudiante --> Estudiante : crea /
modifica

RepositorioEstudiante o-- "0..*" Estudiante
: almacena

note top of EstudianteFacade
Punto de acceso único.
La UI nunca conoce los subsistemas
internos directamente.
end note

@enduml



3.5. Ventajas y desventajas

- **Ventajas:**
 - Simplicidad para el cliente: La UI (FormularioCrudEstudiante) solo necesita conocer cuatro métodos. No importa cuántos subsistemas haya detrás; el formulario siempre hace lo mismo: llamar a EstudianteFacade.
 - Bajo acoplamiento: Si mañana se cambia RepositorioEstudiante para usar una base de datos real en lugar de memoria, la UI no se modifica. Solo se actualiza el interior de la Facade o el repositorio.
 - Mantenibilidad: Los subsistemas pueden evolucionar de forma independiente. Agregar un nuevo paso (por ejemplo, enviar un correo al agregar un estudiante) solo implica modificar EstudianteFacade, sin tocar la UI.

- Organización clara de responsabilidades: Cada clase tiene un rol delimitado: validar, controlar, persistir. Esto hace el código más legible y testeable por partes.
- Facilita pruebas unitarias: Se puede hacer mock de EstudianteFacade para probar la UI sin depender del repositorio real.
- Desventajas:
 - Riesgo de convertirse en una God class: Si no se diseña con cuidado, la Facade puede acumular demasiada lógica y volverse difícil de mantener ella misma.
 - Capa adicional de indirección: Para sistemas muy simples (como este CRUD de tres campos), añadir una Facade puede parecer sobreingeniería, introduciendo complejidad sin un beneficio proporcional.
 - Puede ocultar demasiado: Si un cliente avanzado necesita acceder a funcionalidades específicas de un subsistema, la Facade puede ser un obstáculo o forzar a saltarla, rompiendo el patrón.
 - No elimina la complejidad interna: La Facade simplifica el acceso, pero los subsistemas siguen siendo complejos. Si el equipo no los conoce, depurar errores internos puede ser difícil.

3.6. Conclusiones y reflexiones

El diseño original ya tenía una separación de responsabilidades sólida (boundary, control, entity, repository), lo que demuestra que una práctica de análisis orientado a objetos. Sin embargo, la UI seguía acoplada directamente al ControlEstudiante, lo que significa que cualquier reestructuración interna de la lógica podría obligar a modificar el formulario.

La adaptación al patrón Facade resuelve precisamente ese punto: introduce EstudianteFacade como un contrato estable entre la interfaz gráfica y los subsistemas. El formulario deja de ser un coordinador implícito y se convierte en un cliente puro que simplemente expresa intenciones ("quiero agregar", "quiero eliminar"), delegando toda la orquestación a la fachada.

En este ejemplo, la Facade es un ejercicio valioso porque enseña a pensar en capas de abstracción y en el impacto que tiene el acoplamiento en la mantenibilidad. En sistemas más grandes donde los subsistemas crecen, se dividen en módulos o se integran con servicios externos, este patrón pasa de ser un ejercicio a una necesidad real. La coherencia entre los diagramas (casos de uso, secuencia, clases) se mantiene intacta, y el patrón encaja de forma natural sobre la arquitectura ya propuesta, lo que confirma que el análisis original fue un punto de partida bien fundamentado.